

Reviewer Pack — Minimal p-adic Valuation of Polynomial Roots vs AC0 Circuit ...

Entry b195b55f798f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:51:28 UTC

Minimal p-adic Valuation of Polynomial Roots vs AC0 Circuit Size

Entry ID: b195b55f798f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:51:28 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Complexity Theory: AC⁰ Circuit Complexity

Statement:

For any polynomial $P(x)$ with integer coefficients and degree d , the minimum p-adic valuation of its roots, denoted as $\min_p \text{val}_p(\rho)$, is upper bounded by a function $f(d)$ such that AC0 circuit size for evaluating $P(x)$ on a Boolean input is $O(f(d))$.

Rationale (proposer's reasoning):

p-adic analysis provides a framework for understanding the arithmetic properties of polynomials in rings of integers. By examining p-adic valuations, we can extract information about the complexity of computing polynomial functions. This connection may reveal new insights into circuit complexity lower bounds.

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d0335c36da7b2b90

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For any polynomial $P(x)$ with integer coefficients and degree d ($1 \leq d \leq 40$), if the minimum p-adic valuation of its roots, $\min_p \text{val}_p(\rho)$, correlates positively with the AC0 circuit size for evaluating $P(x)$ on a Boolean input using at most $f(d)$ gates across 30 random seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic valuation" AND "AC0 circuit complexity" - "polynomial roots" AND "p-adic analysis" AND "circuit size" - "min_p val_p(ρ)" AND "degree d" AND "AC0 circuit complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def p_adic_valuation(n, p):
    if n == 0:
        return float('inf')
    count = 0
    while n % p == 0:
        n //= p
        count += 1
    return count

def min_p_adic_valuation(roots):
    return min(p_adic_valuation(root, 2) for root in roots if root != 0)

def generate_polynomial(d, coeff_range):
    coeffs = [random.randint(coeff_range[0], coeff_range[1]) for _ in range(d + 1)]
    return coeffs

def evaluate_polynomial(poly, x):
    result = 0
    power_of_x = 1
    for coeff in poly:
        result += coeff * power_of_x
        power_of_x *= x
    return result

def construct_ac0_circuit(poly, max_gates):
```

```

n = len(poly) - 1
if n > max_gates:
    return None
circuit = []
for i in range(n):
    circuit.append((poly[i], 'add'))
return circuit

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_trials = 30
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(n_trials):
        d = random.randint(1, 40)
        coeff_range = (-10, 10)
        poly = generate_polynomial(d, coeff_range)
        roots = []
        for x in range(-10, 11):
            if evaluate_polynomial(poly, x) == 0:
                roots.append(x)

        val_p = min_p_adic_valuation(roots)
        circuit = construct_ac0_circuit(poly, d)
        if circuit is None:
            conjecture_holds = False
            counterexample = "AC0 circuit size exceeds max_gates"
            break

        mean_circuit_size = sum(len(gate) for gate in circuit) / len(circuit)
        total_metric_value += val_p * mean_circuit_size
        instances_tested += 1

    if conjecture_holds:
        return {
            "metric_name": "min_p_adic_valuation * mean_circuit_size",
            "metric_value": total_metric_value / instances_tested,
            "instances_tested": instances_tested,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "min_p_adic_valuation * mean_circuit_size",
            "metric_value": total_metric_value / instances_tested,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": counterexample
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

```

```

results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")

    if not trial_result["conjecture_holds"]:
        break

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
    print(f"RESULT: FALSIFIED counterexample=\"AC0 circuit size exceeds max_gates\" first_fail
else:
    print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dba9e47e.py", line 107, in <module>
    trial_result = run_trial(seed)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dba9e47e.py", line 73, in run_trial
    val_p = min_p_adic_valuation(roots)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dba9e47e.py", line 33, in min_p_adic_va
    return min(p_adic_valuation(root, 2) for root in roots if root != 0)
    ~~~~~
ValueError: min() iterable argument is empty

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete successfully to provide a result for the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14075
2	preregistration	ollama_remote	glm4:latest	0	0	9963
3	novelty	ollama_remote	glm4:latest	0	0	9347
4	novelty	ollama_remote	glm4:latest	0	0	9216
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12247
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11711
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12953
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11381
9	judge	ollama_remote	glm4:latest	0	0	12568

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103462 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b195b55f798f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b195b55f798f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b195b55f798f.tar.gz` (if generated)